Chapter 8 | Worked solutions with Python

From the Original Block to Modern Decoders

These solutions cover all 14 exercises in chapter order. Exercises 1-6 develop the calculations by hand, 7-11 connect them to the decoder, and 12-14 are optional investigations. Try each problem before reading its solution. The written PDF contains every explanation and result; the notebook PDF adds the executable checks and their saved outputs. The last exercise is an experimental plan, not a claim about measured model quality.

All calculations use local examples on CPU. No dataset or pretrained model download is needed. The saved run uses Python 3.12.14, PyTorch 2.15.0, and NumPy 2.4.6. Floating-point results can vary slightly with software and hardware. The equality checks state absolute tolerances and use zero relative tolerance.

Open this notebook inside `transformers-book` with the packages from `requirements-ch05.txt`. Restart the kernel and run all cells in order. We use one CPU thread and float64 for small numerical comparisons.

In [1]

```
from pathlib import Path
import sys

import torch
from torch import nn
from torch.nn import functional as F

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch08_modern" / "model.py"
    )
    if model_path.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )

torch.set_num_threads(1)
```

In [2]

```
import math
from examples.ch08_modern.model import rms_norm, rotate_pairs

torch.set_printoptions(precision=9, linewidth=90)
DTYPE = torch.float64
ATOL = 1e-12
print("PyTorch", torch.__version__, "| CPU float64 | one thread")
```

Output

```
PyTorch 2.15.0 | CPU float64 | one thread
```

1. Move the normalizer

The residual branch carries the input $x = (3, 4)$. The other branch computes an update, here always $(0, 0)$. The only difference between the two arrangements is where LayerNorm acts.

For this input, the mean is $(3+4)/2 = 3.5$. Subtracting it gives $(-0.5, 0.5)$. The variance uses the mean of squared deviations, so it is $(0.25 + 0.25)/2 = 0.25$. Its square root is 0.5. With the stated unit gains, zero offsets, and zero epsilon, LayerNorm returns $(-1, 1)$.

Placement	What the update branch reads	Residual sum	Final output
Post-norm	$(3, 4)$	$(3, 4) + (0, 0)$	$(-1, 1)$ after LayerNorm
Pre-norm	$(-1, 1)$	$(3, 4) + (0, 0)$	$(3, 4)$

In pre-norm the normalized copy goes to the update branch. The bypass still carries the original input to the addition. A zero update therefore leaves the state unchanged. In post-norm, the sum still goes through normalization. The zero epsilon here is safe because the chosen vector has nonzero variance; real implementations use a positive epsilon.

In [3]

```
x = torch.tensor([3., 4.], dtype=DTYPE)
ln = lambda z: F.layer_norm(z, (2,), eps=0.)
zero = lambda z: torch.zeros_like(z)
post = ln(x + zero(x))
pre = x + zero(ln(x))
torch.testing.assert_close(post, x.new_tensor([-1., 1.]), atol=ATOL, rtol=0)
torch.testing.assert_close(pre, x, atol=0, rtol=0)
print("Post-norm:", post.tolist())
print("Pre-norm: ", pre.tolist())
```

Output

```
Post-norm: [-1.0, 1.0]
Pre-norm:  [3.0, 4.0]
```

2. Rescale the same token

For $(3, 4)$, the mean square is $s = (9 + 16)/2 = 12.5$. Divide by its square root before applying the feature gains $(2, 1/2)$:

$$(2, 1/2) \odot \frac{(3, 4)}{\sqrt{12.5}} = \frac{(6, 2)}{\sqrt{12.5}} \approx (1.697056, 0.565685).$$

For $(6, 8)$, $s = (36 + 64)/2 = 50$. The gained output is $(12, 4)/\sqrt{50} = (6, 2)/\sqrt{12.5}$ again. Doubling the vector doubles its RMS, so the factors cancel. This is exact for positive rescaling when epsilon is zero and the input is nonzero. With positive epsilon it is generally approximate.

Before gains, the mean square is one and the length is $\sqrt{2}$. After gains, the mean square is $(36+4)/(2 \cdot 12.5) = 1.6$. Each gain changes its coordinate's contribution. RMSNorm does not constrain the gained output to unit RMS.

Now add ten to each original coordinate. LayerNorm centers $(13, 14)$ to $(-0.5, 0.5)$, just as it centers $(3, 4)$, and returns $(-1, 1)$ in both cases. RMSNorm gives $(13, 14)/\sqrt{182.5} \approx (0.962303, 1.036326)$ with unit gains.

It preserves the direction of each input before gains; a shared additive shift changes that direction. LayerNorm removes the common additive shift.

In [4]

```
rows = torch.tensor([[3., 4.], [6., 8.]], dtype=DTYPE)
gains = torch.tensor([2., .5], dtype=DTYPE)
gained = rms_norm(rows, gains, eps=0.)
torch.testing.assert_close(gained[0], gained[1], atol=ATOL, rtol=0)
unit = rms_norm(rows, torch.ones(2, dtype=DTYPE), eps=0.)
print("Gained outputs:\n", gained)
print("Gained mean squares:", gained.square().mean(-1).tolist())
print("Unit-gain lengths:  ", unit.norm(dim=-1).tolist())
shift_rows = torch.tensor([[3., 4.], [13., 14.]], dtype=DTYPE)
print("LayerNorm, shifted: \n", F.layer_norm(shift_rows, (2,), eps=0.))
print("RMSNorm, shifted:   \n", rms_norm(shift_rows, torch.ones(2), eps=0.))
```

Output

```
Gained outputs:
 tensor([[1.697056275, 0.565685425],
        [1.697056275, 0.565685425]], dtype=torch.float64)
Gained mean squares: [1.5999999999999999, 1.5999999999999999]
Unit-gain lengths:   [1.4142135623730951, 1.4142135623730951]
LayerNorm, shifted:
 tensor([[-1.,  1.],
        [-1.,  1.]], dtype=torch.float64)
RMSNorm, shifted:
 tensor([[0.848528137, 1.131370850],
        [0.962303173, 1.036326494]], dtype=torch.float64)
```

3. Change the sign of a feature

Apply SiLU to each gate coordinate, then multiply by the corresponding up feature. These multipliers are not probabilities: one is negative and one exceeds one.

Gate input	SiLU value	Up feature	Product
-1	$-1/(1+e) \approx -0.268941$	3	$a \approx -0.806824$
2	$2/(1+e^{-2}) \approx 1.761594$	-1	$b \approx -1.761594$

The down projection is

$$W_{\text{down}} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}.$$

The product (a, b) weights the two rows of this matrix. Therefore $(a, b)W_{\text{down}} = (a, 0, a, 0) + (0, b, 0, -b) = (a, b, a, -b)$, or approximately $(-0.806824, -1.761594, -0.806824, 1.761594)$. The first feature changes sign because its activated gate is negative. The fourth output is positive because the second negative product is multiplied by -1 in the down projection.

In [5]

```
g = torch.tensor([-1., 2.], dtype=DTYPE)
u = torch.tensor([3., -1.], dtype=DTYPE)
down = torch.tensor([[1., 0., 1., 0.], [0., 1., 0., -1.]], dtype=DTYPE)
product = F.silu(g) * u
a, b = -3 / (1 + math.e), -2 / (1 + math.exp(-2))
expected = g.new_tensor([a, b, a, -b])
torch.testing.assert_close(product @ down, expected, atol=ATOL, rtol=0)
print("SiLU:", F.silu(g).tolist())
print("Product:", product.tolist())
print("Output:", (product @ down).tolist())
```

Output

```
SiLU: [-0.2689414213699951, 1.7615941559557646]
Product: [-0.8068242641099853, -1.7615941559557646]
Output: [-0.8068242641099853, -1.7615941559557646, -0.8068242641099853, 1.7615941559557646]
```

4. Match a parameter budget

The conventional network has matrices of sizes $d \times f$ and $f \times d$, so it has $2df$ weights. SwiGLU has two matrices of size $d \times m$ and one of size $m \times d$, giving $3dm$. No biases are included. Setting the counts equal gives $m = 2f/3$.

Model width	Conventional width	Conventional weights	Ideal gated width	Chosen gated width	Gated weights
12	48	1,152	32	32	1,152
10	40	800	$80/3$	32	960

The first count matches exactly. In the second case, the ideal width is about 26.67; rounding upward to a multiple of eight gives 32. The excess is 160 weights, or 20 percent of the conventional count. A hardware-friendly width constraint can therefore prevent an exact parameter match. Matching counts does not guarantee identical measured runtime.

In [6]

```
for d, f in ((12, 48), (10, 40)):
    ideal = 2 * f / 3
    rounded = 8 * math.ceil(ideal / 8)
    conventional, gated = 2 * d * f, 3 * d * rounded
    print(f"d={d}, f={f}: ideal m={ideal:.6f}, rounded m={rounded}")
    print(f"  conventional={conventional}, gated={gated}, excess={gated-conventional}")
assert 3 * 12 * 32 == 1152
assert 3 * 10 * 32 == 960
```

Output

```
d=12, f=48: ideal m=32.000000, rounded m=32
  conventional=1152, gated=1152, excess=0
d=10, f=40: ideal m=26.666667, rounded m=32
  conventional=800, gated=960, excess=160
```

5. Check the sign in RoPE

The horizontal query is $q = (1, 0)^\top$ and the vertical key is $k = (0, 1)^\top$. Use $R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$. At $m = 3$, the query turns by $\pi/2$ and becomes $(0, 1)^\top$. At $n = 1$, the key turns by $\pi/6$ and becomes $(-1/2, \sqrt{3}/2)^\top$. Their dot product is $\sqrt{3}/2 \approx 0.866025$.

Moving the query's rotation to the other side of the dot product transposes it:

$$(R(m\omega)q)^\top R(n\omega)k = q^\top R(-m\omega)R(n\omega)k = q^\top R((n-m)\omega)k.$$

Here the relative angle is $-\pi/3$. The rotated key is $(\sqrt{3}/2, 1/2)^\top$; the original horizontal query selects its first coordinate. After the shared shift, the query at position 5 becomes $(-\sqrt{3}/2, 1/2)^\top$ and the key at position 3 becomes $(-1, 0)^\top$. Their dot product remains $\sqrt{3}/2$.

The wrong relative angle $+\pi/3$ would give $-\sqrt{3}/2$. Orthogonal content vectors expose this sign error. Equal horizontal vectors would instead select cosine, which is even, so both signs would give $1/2$. A common-shift check alone is therefore not sufficient to validate the sign convention.

In [7]

```
def rotation(angle):
    c, s = math.cos(angle), math.sin(angle)
    return torch.tensor([[c, -s], [s, c]], dtype=DTYPE)

q, k = torch.eye(2, dtype=DTYPE)
omega = math.pi / 6
for m, n in ((3, 1), (5, 3)):
    direct = (rotation(m * omega) @ q) @ (rotation(n * omega) @ k)
    relative = q @ rotation((n - m) * omega) @ k
    wrong = q @ rotation((m - n) * omega) @ k
    torch.testing.assert_close(direct, relative, atol=ATOL, rtol=0)
    torch.testing.assert_close(direct, q.new_tensor(math.sqrt(3)/2), atol=ATOL, rtol=0)
    print(f"m={m}, n={n}: direct={direct:.9f}, relative={relative:.9f}, wrong={wrong:.9f}")
```

Output

```
m=3, n=1: direct=0.866025404, relative=0.866025404, wrong=-0.866025404
m=5, n=3: direct=0.866025404, relative=0.866025404, wrong=-0.866025404
```

6. Build a head from pairs

There are $d_h/2 = 2$ pairs. For pair indices $r = 0, 1$, the frequencies are $10000^{-2r/4}$, giving 1 and 0.01 radians per position. The first frequency turns 100 times as fast as the second. With $n - m = -2$, their relative angles are -2 and -0.02 radians.

Each pair starts as $(1, 0)$ in both query and key, so its contribution is the cosine of its relative angle.

Key position	First contribution	Second contribution	Sum	Scaled logit
$n = 1$	$\cos(-2) \approx -0.416147$	$\cos(-0.02) \approx 0.999800$	0.583653	0.291827
$n = 3$	$\cos(0) = 1$	$\cos(0) = 1$	2	1

Scaling divides the sum by $\sqrt{d_h} = 2$. The head vector has length $\sqrt{2}$, which explains why its unscaled dot product with itself is two. Neither number in the last column is an attention probability. Softmax must compare a complete row of permitted keys. RoPE also produces finite scores for future keys; the causal mask must exclude those keys before softmax.

In [8]

```
from examples.ch08_modern.run_examples import pair_score_example

pair_report = pair_score_example()
expected_pairs = torch.tensor([math.cos(2), math.cos(.02)], dtype=DTYPE)
torch.testing.assert_close(
    torch.tensor(pair_report["pair_contributions"], dtype=DTYPE),
    expected_pairs, atol=ATOL, rtol=0,
)
content = torch.tensor([[[[1., 0., 1., 0.]]]], dtype=DTYPE)
q3 = rotate_pairs(content, torch.tensor([3]))
for n in (1, 3, 4):
    kn = rotate_pairs(content, torch.tensor([n]))
    contributions = (q3 * kn).reshape(2, 2).sum(-1)
    print(f"n={n}: pairs={contributions.tolist()}, logit={contributions.sum()/2:.9f}")
assert abs(pair_report["scaled_logit"] - (math.cos(2)+math.cos(.02))/2) < ATOL
```

Output

```
n=1: pairs=[-0.41614683654714246, 0.9998000066665778], logit=0.291826585
n=3: pairs=[0.9999999999999999, 1.0], logit=1.000000000
n=4: pairs=[0.5403023058681397, 0.9999500004166654], logit=0.770126153
```

7. Inspect a small denominator

Take a two-feature token $x = (3 \cdot 10^{-4}, 4 \cdot 10^{-4})$. RMSNorm divides both entries by a shared scale computed from this token alone. Let s be the average of the two squared entries. The denominator is $\sqrt{s + \epsilon}$, with $\epsilon = 10^{-6}$. The unit gains multiply each coordinate by one after the division.

If $y_i = x_i/\sqrt{s + \epsilon}$, then both y_i^2 contain the same factor $1/(s + \epsilon)$. Take that factor outside the average to obtain the output mean square.

$$\frac{y_1^2 + y_2^2}{2} = \frac{(x_1^2 + x_2^2)/2}{s + \epsilon} = \frac{s}{s + \epsilon}.$$

For the first row, the input mean square is $(9 + 16)10^{-8}/2 = 1.25 \cdot 10^{-7}$. The value under the square root is $1.125 \cdot 10^{-6}$. Epsilon is eight times the input mean square, so it makes a substantial contribution to the denominator.

$$\frac{1.25 \cdot 10^{-7}}{1.125 \cdot 10^{-6}} = \frac{1}{9} \approx 0.111111.$$

Now multiply the first row by ten to get $(3 \cdot 10^{-3}, 4 \cdot 10^{-3})$. Squaring the entries multiplies their mean square by 100. Epsilon stays fixed, giving

$$\frac{1.25 \cdot 10^{-5}}{1.25 \cdot 10^{-5} + 10^{-6}} = \frac{25}{27} \approx 0.925926.$$

The zero row has $s = 0$, and its denominator is $\sqrt{10^{-6}} = 10^{-3}$. Dividing each zero coordinate by this positive number leaves zero. In Python, compute one mean square per row by reducing the feature axis with `mean(dim=-1)`. Compare with the fractions above using float64 on CPU, an absolute tolerance of 10^{-12}, and zero relative tolerance.

In [9]

```
from examples.ch08_modern.model import rms_norm

rms_input = torch.tensor(
    [[3e-4, 4e-4], [0.0, 0.0]], dtype=torch.float64
)
rms_scaled = rms_input.clone()
rms_scaled[0] *= 10
gains = torch.ones_like(rms_input[0])
epsilon = 1e-6
rms_atol = 1e-12

rms_cases = (
    ("original", rms_input, 1 / 9),
    ("first row x10", rms_scaled, 25 / 27),
)
for name, rows, expected in rms_cases:
    normalized = rms_norm(rows, gains, eps=epsilon)
    mean_squares = normalized.square().mean(dim=-1)
    torch.testing.assert_close(
        mean_squares, rows.new_tensor([expected, 0.0]),
        atol=rms_atol, rtol=0,
    )
    assert normalized.dtype == torch.float64
    assert torch.isfinite(normalized[1]).all()
    assert torch.count_nonzero(normalized[1]).item() == 0
    print(name)
    print(f"  input mean square (row 0)   {rows[0].square().mean():.9g}")
    print(f"  output mean square (row 0)  {mean_squares[0]:.12f}")
    print(f"  zero row output             {normalized[1].tolist()}")
```

Output

```
original
  input mean square (row 0)   1.25e-07
  output mean square (row 0)  0.111111111111
  zero row output             [0.0, 0.0]
first row x10
  input mean square (row 0)   1.25e-05
  output mean square (row 0)  0.925925925926
  zero row output             [0.0, 0.0]
```

The scaled row moves closer to mean square one because s now dominates ϵ. It is still below one because the added ϵ is positive. The output depends on the ratio ϵ/s for a nonzero row. The second row gives the same finite pair of zeros in both calls, since the scale calculation runs independently for each token.

8. Rebuild the worked gate

The chapter sends a width-four token through two separate projections. The gate branch should produce $g = (0, 2)$ and the feature branch should produce $u = (3, -1)$ on the input $x = (1, 0, 0, 0)$. With this input, each output coordinate of `nn.Linear` is just the weight attached to the first input coordinate. That lets us set both projections by hand.

PyTorch stores a linear layer's weight with one row per output coordinate and one column per input coordinate. It evaluates $xA^\top$ for a stored weight A. The gate and up weights are therefore each 2×4. Zero both matrices, then set the first column of the gate matrix to $(0, 2)^\top$ and the first column of the up matrix to $(3, -1)^\top$.

The chapter writes its down projection on the right of a two-coordinate row vector. Its mathematical matrix is

$$W_{\text{down}} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}.$$

PyTorch needs four output rows and two input columns for `nn.Linear(2, 4)`. Copy $W_{\text{down}}^{\top}$ into the weight so that the linear layer computes the intended row-vector multiplication. Use `torch.no_grad()` while changing the weights in place. These assignments set the parameters and do not form part of a differentiable calculation.

In [10]

```
from examples.ch08_modern.model import SwiGLU

gate_layer = SwiGLU(4, 2).double()
gate_input = torch.tensor(
    [1.0, 0.0, 0.0, 0.0], dtype=torch.float64
)
down_matrix = gate_input.new_tensor([
    [1.0, 0.0, 1.0, 0.0],
    [0.0, 1.0, 0.0, -1.0],
])
with torch.no_grad():
    gate_layer.gate_proj.weight.zero_()
    gate_layer.up_proj.weight.zero_()
    gate_layer.gate_proj.weight[:, 0] = gate_input.new_tensor([0, 2])
    gate_layer.up_proj.weight[:, 0] = gate_input.new_tensor([3, -1])
    gate_layer.down_proj.weight.copy_(down_matrix.T)

print("gate weight", gate_layer.gate_proj.weight.tolist())
print("up weight  ", gate_layer.up_proj.weight.tolist())
print("down weight", gate_layer.down_proj.weight.tolist())
```

Output

```
gate weight [[0.0, 0.0, 0.0, 0.0], [2.0, 0.0, 0.0, 0.0]]
up weight   [[3.0, 0.0, 0.0, 0.0], [-1.0, 0.0, 0.0, 0.0]]
down weight [[1.0, 0.0], [0.0, 1.0], [1.0, 0.0], [0.0, -1.0]]
```

The SiLU function multiplies its input by its sigmoid, $\mathrm{SiLU}(a) = a/(1+\exp(-a))$. Define $a_2 = 2/(1+\exp(-2))$. The activated gate is $(0, a_2)$, with $a_2 \approx 1.761594155956$. Multiplying by the up features coordinate by coordinate gives $(0 \cdot 3, a_2 \cdot (-1)) = (0, -a_2)$.

The product's first coordinate weights the first row of W_{down} by zero. The second coordinate weights the second row by $-a_2$. The update is $(0, -a_2, 0, a_2)$. In particular, the positive fourth coordinate comes from multiplying the negative product by the matrix entry -1. The following check computes a_2 separately with the scalar formula and compares the intermediate tensors and full module output in CPU float64 with absolute tolerance 10^{-12} and zero relative tolerance.

In [11]

```
from math import exp

gate_layer.eval()
with torch.inference_mode():
    gate_values = gate_layer.gate_proj(gate_input)
    up_values = gate_layer.up_proj(gate_input)
    gate_active = F.silu(gate_values)
    gate_product = gate_active * up_values
    gate_output = gate_layer(gate_input)

a2 = 2 / (1 + exp(-2))
gate_stages = (
    ("gate projection", gate_values, [0, 2]),
    ("up projection", up_values, [3, -1]),
    ("activated gate", gate_active, [0, a2]),
    ("elementwise product", gate_product, [0, -a2]),
    ("module output", gate_output, [0, -a2, 0, a2]),
)
for name, actual, expected in gate_stages:
    torch.testing.assert_close(
        actual, gate_input.new_tensor(expected), atol=1e-12, rtol=0
    )
    numbers = ", ".join(f"{value:.12g}" for value in actual)
    print(f"{name:<19} [{numbers}]")
```

Output

```
gate projection     [0, 2]
up projection       [3, -1]
activated gate      [0, 1.76159415596]
elementwise product [0, -1.76159415596]
module output       [0, -1.76159415596, 0, 1.76159415596]
```

9. Probe the attention mask

The two inputs begin with the same three tokens. The final token is `stopped` in the first sequence and `moved` in the second. Each output row holds 17 logits, meaning the decoder's scores for the 17 possible next tokens. The row at index 2 reads up through the shared third token and predicts a possible token at index 3. The observed token at index 3 is outside that row's allowed inputs.

Create a single model after setting seed 81, keep its weights fixed, and use evaluation mode. Run the inputs as integer IDs (`torch.long`) with a batch axis. The model's weights and logits use CPU float64. Compare the first three rows, then measure the difference at row 3. For the padding comparison, append `PAD=0` to the first sequence and compare just its four valid rows. The output shape changes from $[1, 4, 17]$ to $[1, 5, 17]$ when a position is appended.

Each reported maximum is the largest absolute change across the selected rows and all vocabulary coordinates. Apply `torch.testing.assert_close` to both invariance checks with `rtol=0` and `atol=1e-10`. In this check every logit can differ by at most 10^{-10}.

In [12]

```
from examples.ch08_modern.model import ModernDecoder

with torch.random.fork_rng(devices=[]):
    torch.manual_seed(81)
    mask_model = ModernDecoder().double().eval()

base_ids = torch.tensor([[7, 8, 9, 10]], dtype=torch.long)
changed_ids = torch.tensor([[7, 8, 9, 16]], dtype=torch.long)
padded_ids = torch.tensor([[7, 8, 9, 10, 0]], dtype=torch.long)

with torch.inference_mode():
    base_logits = mask_model(base_ids)
    changed_logits = mask_model(changed_ids)
    padded_logits = mask_model(padded_ids)

mask_atol = 1e-10
torch.testing.assert_close(
    base_logits[:, :3], changed_logits[:, :3], rtol=0, atol=mask_atol
)
torch.testing.assert_close(
    base_logits, padded_logits[:, :4], rtol=0, atol=mask_atol
)

edit_delta = (changed_logits - base_logits).abs()
pad_delta = (padded_logits[:, :4] - base_logits).abs()
print("Base and padded shapes", base_logits.shape, padded_logits.shape)
print(f"Edit, first three rows   {edit_delta[:, :3].max():.12g}")
print(f"Edit, final row          {edit_delta[:, 3].max():.12g}")
print(f"Append PAD, valid rows   {pad_delta.max():.12g}")
```

Output

```
Base and padded shapes torch.Size([1, 4, 17]) torch.Size([1, 5, 17])
Edit, first three rows   0
Edit, final row          1.69309613396
Append PAD, valid rows   0
```

In the saved run, changing the final token alters its own logit row. Its embedding changes at the start, and the causal mask permits the final query to use the key at its own position. A difference is possible there. For these weights the maximum change is about 1.6931. The first three rows agree because every block masks later keys. The first block starts with identical embeddings at those three positions and can only draw their updates from earlier or current positions. The same argument applies in the next block. RMSNorm, the feed-forward network, and the vocabulary projection process positions individually, so those operations preserve the agreement.

Appending PAD preserves the positions and contents of the four real tokens. To inspect the two mask rules, read the Boolean grid for the padded input. Rows are queries and columns are keys. An entry of 1 allows that query to read the key. The mask combines the lower triangle of causal permissions with the validity of each key. The PAD column is zero even in the final row, where the causal rule alone would allow the diagonal.

In [13]

```
from examples.ch05_transformer.model import build_masks

valid_tokens = padded_ids != mask_model.pad_id
_, allowed_keys = build_masks(valid_tokens, valid_tokens)
print("Validity mask", valid_tokens[0].tolist())
print("Allowed keys, one row per query")
print(allowed_keys[0, 0].to(torch.int64))
assert not allowed_keys[..., -1].any()
causal_only = torch.ones(5, 5, dtype=torch.bool).tril()
print("PAD diagonal, full mask:", allowed_keys[0, 0, -1, -1].item())
print("PAD diagonal, causal only:", causal_only[-1, -1].item())
assert not allowed_keys[0, 0, -1, -1]
assert causal_only[-1, -1]
```

Output

```
Validity mask [True, True, True, True, False]
Allowed keys, one row per query
tensor([[1, 0, 0, 0, 0],
        [1, 1, 0, 0, 0],
        [1, 1, 1, 0, 0],
        [1, 1, 1, 1, 0],
        [1, 1, 1, 1, 0]])
PAD diagonal, full mask: False
PAD diagonal, causal only: True
```

For the first four queries, the newly appended key is already later than the query and therefore excluded by causality. The invariance check shows that adding right-padding preserves these particular outputs. Inspecting the full mask also checks the separate rule that a padding key is invalid. The model can still produce a logit row at the padded query position. A padded label is set to `IGNORE` in the loss calculation, as the next exercise shows.

10. Check a full run

Run `python -m examples.ch08_modern.run_examples` from the repository root. It prints a JSON record with component checks and a decoder example. The notebook invokes the same command with the current kernel's Python executable and displays the relevant fields from the actual run.

The decoder gets a batch of two examples. Shifting *the red car stopped* supplies three input and label pairs. Shifting *the blue moved* supplies two. The example pads the input of the second row to length three and adds an `IGNORE` entry, encoded as `-100`, to its label row. The shape of the scores is $[2, 3, 17]$. It records two sequences, three output positions per sequence, and 17 vocabulary scores at each position. There are six output rows in total and five graded labels.

The module uses float64 on CPU for its decoder and runs one SGD update on this fixed batch with learning rate 0.01. It checks that the gradient tensor exists for each parameter and that every entry in each gradient tensor is finite before taking the step.

In [14]

```
import json
import subprocess

run_modern = subprocess.run(
    [sys.executable, "-m", "examples.ch08_modern.run_examples"],
    cwd=repo_root, check=True, capture_output=True, text=True,
)
decoder_report = json.loads(run_modern.stdout)["decoder"]
update_report = decoder_report["update"]

print("Input IDs             ", decoder_report["inputs"])
print("Label IDs             ", decoder_report["labels"])
print("Graded labels         ", decoder_report["scored_labels"])
print("Logits shape          ", decoder_report["logits_shape"])
print("Parameters per block  ", decoder_report["parameters_per_block"])
print("Parameters total      ", decoder_report["parameters_total"])
print("All gradients finite  ", update_report["finite_gradients"])
print(f"Mean loss before       {update_report['loss_before']:.9f}")
print(f"Mean loss after        {update_report['loss_after']:.9f}")

assert decoder_report["logits_shape"] == [2, 3, 17]
assert decoder_report["scored_labels"] == 5
assert update_report["finite_gradients"] is True
```

Output

```
Input IDs             [[7, 8, 9], [7, 15, 0]]
Label IDs             [[8, 9, 10], [15, 16, -100]]
Graded labels         5
Logits shape          [2, 3, 17]
Parameters per block  2592
Parameters total      5744
All gradients finite  True
Mean loss before      3.245773867
Mean loss after       3.197667914
```

The mean loss falls from about 3.245774 to 3.197668 on the batch used for the update. This example checks that the forward pass, the loss, and backpropagation work together on those five label positions. The check for finite gradients allows zero entries, which can arise, for instance, in embedding rows for words absent from the batch.

To derive the parameter count, let $d = 16$ be the residual width, $m = 32$ the intermediate SwiGLU width, and $V = 17$ the vocabulary size. The projections are bias-free, so each linear layer contributes exactly its input width times its output width.

Part of one block	Weight calculation	Parameters
Query, key, value, and attention output projections	$4(16 \cdot 16)$	1,024
SwiGLU gate, up, and down projections	$3(16 \cdot 32)$	1,536
Attention and feed-forward RMSNorm gains	$2(16)$	32
One complete block	$1{,}024 + 1{,}536 + 32$	2,592

The attention projects all 16 coordinates before splitting them into two heads of width eight. That reshaping does not add another factor of two to the four matrix counts. SwiGLU has two $16 \to 32$ projections and one $32 \to 16$ projection. Each RMSNorm has one learned gain per feature. The residual additions, rotary angles, and Boolean masks add no learned parameters.

Outside the blocks, the embedding table contains $17 \cdot 16 = 272$ weights. The untied vocabulary projection has its own $16 \cdot 17 = 272$ weights. The final RMSNorm adds 16. The fixed part of the count is $272 + 272 + 16 = 560$. It stays the same when we change only the number of blocks.

$$P(2) = 560 + 2(2{,}592) = 5{,}744,$$
$$P(3) = 560 + 3(2{,}592) = 8{,}336.$$

The predicted increase is 2,592. Check it by counting the scalar entries of the three-block model's parameter tensors. These are exact integer checks, independent of the weights' floating-point dtype.

In [15]

```
model_width, ff_width, vocabulary_size = 16, 32, 17
block_count = 4 * model_width**2 + 3 * model_width * ff_width
block_count += 2 * model_width
outside_blocks = 2 * vocabulary_size * model_width + model_width

with torch.random.fork_rng(devices=[]):
    three_block_model = ModernDecoder(n_layers=3)
actual_three = sum(p.numel() for p in three_block_model.parameters())
expected_three = outside_blocks + 3 * block_count
two_block_total = decoder_report["parameters_total"]

assert decoder_report["parameters_per_block"] == block_count
assert two_block_total == outside_blocks + 2 * block_count
assert actual_three == expected_three
assert actual_three - two_block_total == block_count

print(f"Two blocks, measured         {two_block_total:,}")
print(f"Three blocks, predicted      {expected_three:,}")
print(f"Three blocks, measured       {actual_three:,}")
print(f"Increase                     {actual_three - two_block_total:,}")
```

Output

```
Two blocks, measured         5,744
Three blocks, predicted      8,336
Three blocks, measured       8,336
Increase                     2,592
```

Within each block, **self-attention mixes positions**. For a query at one position, it forms a weighted sum of value vectors at the allowed positions, so information from other tokens can enter that position's update. The projections that make the queries, keys, and values act on each token separately. It is the attention-weighted sum over keys, `weights @ v`, that combines their information.

SwiGLU expands features within a position. Both the gate and up projections send one token from 16 coordinates to 32. SiLU acts on the gate, the two results multiply feature by feature, and the down projection returns that same token's update to width 16. The down projection lets the result be added to the residual stream at that position.

11. Move the origin, then change the context

Reuse the seed-81 decoder from Exercise 9 without changing its weights. Relabeling positions $[0, 1, 2, 3]$ as $[7, 8, 9, 10]$ adds the same offset to every query and key. Every difference $n - m$ stays the same. With the same embeddings, mask, and fixed frequencies, the first block's scores and outputs agree. The second block receives the same inputs, so the argument repeats. The final RMSNorm and vocabulary projection act on identical vectors.

Prepending token 16 creates a five-token sequence. Among the original four tokens, distances are unchanged, but there is now an additional key and value available to every original query. The softmax includes this new key, which can change how the values are mixed. The shared-shift theorem does not require these logits to remain equal.

Comparison	Largest absolute logit change in the saved run
Shared offset of seven	Less than 10^{-15}
Prepend token 16, align the original four rows	About 0.350693

This checks a positional symmetry of this particular decoder. It neither increases the sequence length in the offset case nor measures predictive quality on unseen long contexts. Added context could happen to leave logits unchanged for special weights; it changes them for this fixed model.

In [16]

```
with torch.inference_mode():
    origin_logits = mask_model(base_ids, start_pos=0)
    shifted_logits = mask_model(base_ids, start_pos=7)
    more_context = mask_model(torch.tensor([[16, 7, 8, 9, 10]]))[:, 1:]
torch.testing.assert_close(origin_logits, shifted_logits, atol=1e-10, rtol=0)
shift_delta = (origin_logits - shifted_logits).abs().max().item()
context_delta = (origin_logits - more_context).abs().max().item()
assert context_delta > 1e-6
print(f"Shared-offset change: {shift_delta:.12g}")
print(f"Added-context change: {context_delta:.12g}")
```

Output

```
Shared-offset change: 4.4408920985e-16
Added-context change: 0.350692604094
```

12. Catch a reset position (optional)

Imagine reading four tokens in two chunks. The first chunk holds tokens 0 and 1. The second holds tokens 2 and 3, although they occupy slots 0 and 1 inside that chunk's tensor. Rotary position embeddings, or RoPE, rotate the features of each query or key using its position in the sequence. When a cache stores a new rotated key, its angle must come from its sequence position. Giving the second chunk the positions 0 and 1 again would repeat the angles already used by the first chunk.

To isolate the effect of position, give all four keys the same unrotated content $(1, 0)$. The tensor has shape $[1, 1, 4, 2]$, meaning one sequence in the batch, one attention head, four tokens, and two features per token. The two features make one pair with frequency 1 radian per position because the first pair uses $\text{base}^0 = 1$. At position n, the rotated key is $(\cos n, \sin n)$. For instance, position 0 keeps the original pair and position 1 turns it to approximately $(0.5403, 0.8415)$.

First rotate the four keys together. Then rotate the first two and the last two separately, using the same positions as in the full sequence. Concatenating with `dim=-2` joins the token axis, which has length 2 in each slice, while preserving the last axis of two features per key. All calculations run in CPU `torch.float64`. The equality checks use absolute tolerance 10^{-12} and relative tolerance 0.

In [17]

```
from examples.ch08_modern.model import rotate_pairs

rope_atol = 1e-12
rope_keys = torch.tensor([1.0, 0.0], dtype=torch.float64)
rope_keys = rope_keys.reshape(1, 1, 1, 2).repeat(1, 1, 4, 1)
rope_positions = torch.arange(4)

rope_full = rotate_pairs(rope_keys, rope_positions)
rope_first = rotate_pairs(
    rope_keys[:, :, :2, :], rope_positions[:2]
)
rope_second = rotate_pairs(
    rope_keys[:, :, 2:, :], rope_positions[2:]
)
rope_split = torch.cat((rope_first, rope_second), dim=-2)

torch.testing.assert_close(
    rope_full, rope_split, atol=rope_atol, rtol=0
)
print("Shape of both versions:", list(rope_full.shape))
print("Max absolute difference:",
      (rope_full - rope_split).abs().max().item())
for position, (x, y) in enumerate(rope_full[0, 0].tolist()):
    print(f"  Key at {position}: ({x:+.9f}, {y:+.9f})")
```

Output

```
Shape of both versions: [1, 1, 4, 2]
Max absolute difference: 0.0
  Key at 0: (+1.000000000, +0.000000000)
  Key at 1: (+0.540302306, +0.841470985)
  Key at 2: (-0.416146837, +0.909297427)
  Key at 3: (-0.989992497, +0.141120008)
```

Each key rotates independently of the others, so the two calculations agree when the position supplied for each key is the same. Now give the second slice the wrong positions $[0, 1]$. The four stored keys then have angles $[0, 1, 0, 1]$ radians even though their sequence positions are $[0, 1, 2, 3]$.

Compare the keys with the query $(1, 0)$ at position $m = 3$. With the same content in each vector, its unscaled dot product with the key rotated at position n is

$$\begin{aligned}\widetilde{\mathbf{q}}_3^\top \widetilde{\mathbf{k}}_n &= \cos 3 \cos n + \sin 3 \sin n \\ &= \cos(n - 3).\end{aligned}$$

The full run therefore gives $[\cos 3, \cos 2, \cos 1, 1]$. In particular, the fourth key shares the query's angle and the score is the dot product of a unit vector with itself. Resetting the second chunk replaces its angular distances from the query, 1 and 0 radians, with 3 and 2 radians. The scores become $[\cos 3, \cos 2, \cos 3, \cos 2]$.

The next cell uses matrix multiplication to compute one dot product per key. Transposing the key tensor's last two axes gives a feature-by-key matrix for each batch item and head. A query row times this matrix produces four unscaled scores in token order.

In [18]

```
rope_reset_second = rotate_pairs(
    rope_keys[:, :, 2:, :], rope_positions[:2]
)
rope_reset = torch.cat((rope_first, rope_reset_second), dim=-2)
rope_query = rotate_pairs(
    rope_keys[:, :, :1, :], rope_positions[3:]
)

rope_full_scores = (rope_query @ rope_full.transpose(-2, -1))
rope_split_scores = (rope_query @ rope_split.transpose(-2, -1))
rope_reset_scores = (rope_query @ rope_reset.transpose(-2, -1))

expected_full = (rope_positions.to(torch.float64) - 3).cos()
expected_reset = torch.tensor(
    [-3.0, -2.0, -3.0, -2.0], dtype=torch.float64
).cos()
for actual, expected in (
    (rope_full_scores, expected_full),
    (rope_split_scores, expected_full),
    (rope_reset_scores, expected_reset),
):
    torch.testing.assert_close(
        actual[0, 0, 0], expected, atol=rope_atol, rtol=0
    )

for name, scores in (
    ("All four together", rope_full_scores),
    ("Split with sequence indices", rope_split_scores),
    ("Split with reset", rope_reset_scores),
):
    values = ", ".join(f"{v:+.7f}" for v in scores[0, 0, 0])
    print(f"{name:27s} [{values}]")
```

Output

```
All four together           [-0.9899925, -0.4161468, +0.5403023, +1.0000000]
Split with sequence indices [-0.9899925, -0.4161468, +0.5403023, +1.0000000]
Split with reset            [-0.9899925, -0.4161468, -0.9899925, -0.4161468]
```

The first two scores stay the same because those keys keep their original angles. In the correct run, the query is close in angle to key 2 and aligned with key 3. After the reset, key 2 receives key 0's score and key 3 receives key 1's score. We held the content fixed, so this change comes entirely from the wrong position indices.

A cache can use the starting sequence index of a chunk plus the index within that chunk. For this example, the starting indices are 0 and 2, and a single new query uses the index 3 for the fourth token. RoPE's common-shift property still holds if all query and key positions move by the same amount. Here the query and the first two keys stay at their original positions while only the last two keys move. This probe checks the position calculation for rotated keys and the resulting dot products.

13. Isolate a residual gradient (optional)

Use $x = (1, 2, 4)$ and an update that is identically zero. Pre-norm returns $y = x$. Post-norm returns $y = \mathrm{LayerNorm}(x)$. Here LayerNorm has unit gains, zero offsets, and $\epsilon = 10^{-6}$; its mean is $7/3$ and variance is $14/9$. Use a fresh differentiable tensor for each loss and placement.

For the summed loss, pre-norm gives $L = x_1 + x_2 + x_3$, so its input gradient is $(1, 1, 1)$. In post-norm, all centered coordinates sum to zero and share one denominator. Thus $L = 0$ for every input and its gradient is zero.

Now use $L = y_1 - y_2$. Pre-norm has gradient $(1, -1, 0)$. Post-norm no longer sums all the centered coordinates, so the cancellation does not apply. With $c = x - \mu$, $r = \sqrt{\mathrm{mean}(c^2) + \epsilon}$, and $a = (1, -1, 0)$, differentiating $L = a^\top c / r$ gives

$$\nabla_x L = \frac{a - \mathrm{mean}(a)}{r} - \frac{c\,(a^\top c)}{3r^3}.$$

The first term differentiates the centered numerator. The second accounts for the denominator changing when the input changes. Substitution gives approximately $(0.572703, -0.859054, 0.286351)$, which is nonzero.

Loss	Pre-norm input gradient	Post-norm input gradient
$\sum_i y_i$	$(1, 1, 1)$	$(0, 0, 0)$
$y_1 - y_2$	$(1, -1, 0)$	$(0.572703, -0.859054, 0.286351)$

The residual branch contributes the incoming gradient directly in pre-norm. Post-norm transforms it through the normalization Jacobian. The first loss cannot establish that post-norm always blocks gradients. We use three features and positive epsilon here to avoid the additional degeneracy of two-feature, zero-epsilon LayerNorm: its output is locally $(-1, 1)$ or $(1, -1)$, so even many other losses would have zero input gradient.

In [19]

```
def placed_output(x, placement):
    normalized = lambda v: F.layer_norm(v, (3,), eps=1e-6)
    if placement == "pre":
        return x + torch.zeros_like(normalized(x))
    return normalized(x + torch.zeros_like(x))

gradient_results = {}
for loss_name, weights in (("sum", [1., 1., 1.]), ("difference", [1., -1., 0.])):
    for placement in ("pre", "post"):
        x = torch.tensor([1., 2., 4.], dtype=DTYPE, requires_grad=True)
        a = x.new_tensor(weights)
        loss = (placed_output(x, placement) * a).sum()
        (gradient,) = torch.autograd.grad(loss, x)
        gradient_results[loss_name, placement] = gradient
        print(loss_name, placement, gradient.tolist())
torch.testing.assert_close(gradient_results["sum", "pre"], torch.ones(3, dtype=DTYPE))
torch.testing.assert_close(gradient_results["sum", "post"], torch.zeros(3, dtype=DTYPE), atol=ATOL, rtol=0)
torch.testing.assert_close(gradient_results["difference", "pre"], a, atol=ATOL, rtol=0)
```

Output

```
sum pre [1.0, 1.0, 1.0]
sum post [-1.1102230246251565e-16, -1.1102230246251565e-16, 1.1102230246251565e-16]
difference pre [1.0, -1.0, 0.0]
difference post [0.5727026244242565, -0.8590536789204358, 0.28635105449617926]
```

Check the nonzero result by a separate numerical method. For coordinate j, evaluate the loss at $x + he_j$ and $x - he_j$, keeping other coordinates fixed. The quotient $[L(x + he_j) - L(x - he_j)]/(2h)$ estimates the derivative. With $h = 10^{-5}$, the analytic, automatic, and finite-difference gradients agree within absolute tolerance 10^{-8} on CPU float64.

In [20]

```
x = torch.tensor([1., 2., 4.], dtype=DTYPE)
a = x.new_tensor([1., -1., 0.])
centered = x - x.mean()
r = (centered.square().mean() + 1e-6).sqrt()
analytic = (a - a.mean()) / r - centered * (a @ centered) / (3 * r**3)
h = 1e-5
finite_difference = []
for direction in torch.eye(3, dtype=DTYPE):
    plus = (placed_output(x + h*direction, "post") * a).sum()
    minus = (placed_output(x - h*direction, "post") * a).sum()
    finite_difference.append((plus-minus)/(2*h))
finite_difference = torch.stack(finite_difference)
automatic = gradient_results["difference", "post"]
torch.testing.assert_close(automatic, analytic, atol=ATOL, rtol=0)
torch.testing.assert_close(automatic, finite_difference, atol=1e-8, rtol=0)
print("Analytic:", analytic.tolist())
print("Finite difference:", finite_difference.tolist())
print("Maximum finite-difference error:", (automatic-finite_difference).abs().max().item())
```

Output

```
Analytic: [0.5727026244242566, -0.8590536789204358, 0.2863510544961792]
Finite difference: [0.5727026244184419, -0.8590536789199187, 0.28635105448482356]
Maximum finite-difference error: 1.1355694162773489e-11
```

14. Design a matched comparison (optional)

The experimental question is whether replacing a ReLU feed-forward sublayer with SwiGLU changes next-token validation loss under a stated budget. A fair comparison fixes the rest of the architecture and makes the gated width smaller to pay for its third matrix. Here is one concrete protocol; other well-specified choices are acceptable.

Choice	Fixed plan
Shared architecture	Two blocks, $d = 16$, two attention heads, pre-RMSNorm, RoPE, untied readout, no dropout
ReLU feed-forward	Bias-free width $f = 48$: $2 \cdot 16 \cdot 48 = 1{,}536$ weights per block
SwiGLU feed-forward	Bias-free width $m = 32$: $3 \cdot 16 \cdot 32 = 1{,}536$ weights per block
Training	3,000 updates, batches of 32 sentences, fresh AdamW with learning rate $3 \cdot 10^{-4}$, betas $(0.9, 0.999)$, weight decay 0.01
Paired seeds	101-105; identical shared initial tensors and batch order within each pair; independently initialized feed-forward maps
Precision	Float32 training on one CPU thread; record all software versions

Use a reproducible synthetic corpus so the data are available to every student. Fix 25 subjects, 20 objects, and 30 places. Assign each subject one of five verbs by its list index modulo five. This gives $25 \cdot 20 \cdot 30 = 15{,}000$ scenes. Render each scene in two orders, such as *the fox finds the bell near the gate* and *today near the gate the fox finds the bell*. Save the word lists and generator.

Shuffle scene IDs once with seed 807. Split into 12,000 training, 1,500 validation, and 1,500 test scenes before rendering. Both versions of a scene stay in its split, preventing the same scene from leaking through another word order. Fit one vocabulary on training data, verify coverage, and share it between models. Add BOS and EOS before making shifted input-target pairs; exclude padding labels from the loss. This small task can reveal differences on the constructed distribution, not establish general language ability.

Copy identical initial embedding, attention, normalization, and output tensors into both models for each paired seed. Initialize each feed-forward linear map uniformly on $[-1/\sqrt{f_{\text{in}}}, 1/\sqrt{f_{\text{in}}}]$, where f_{in} is that map's input width. Use the same minibatches within a pair, so both models see the same number of valid targets, as well as the same number of updates. Record the exact total parameter counts.

Measure validation loss every 250 updates. Precommit to the checkpoint at update 3,000 for the comparison, then evaluate each final model on the test split once. If hyperparameters are tuned instead, give both architectures the same search budget and keep test data untouched until choices are final. Aggregate total negative log probability and total valid-token count:

$$\text{NLL}_{\text{token}} = \frac{\sum_b \sum_{t \in \mathcal{V}_b} -\log p_\theta(y_{b,t} \mid x_{b,\le t})}{\sum_b |\mathcal{V}_b|}.$$

The score is in nats per valid token. A simple mean of batch means would weight short and long batches incorrectly. For each seed, report the paired difference SwiGLU minus ReLU, then its mean and spread across seeds. A negative difference favors SwiGLU on this corpus and budget. Do not invent a winner before running the experiment.

Benchmark execution time separately. Fix device, precision, threads, batch shapes, and software. Keep a representative batch on the device, warm up for 25 steps, then time ten intervals of 100 forward-backward-optimizer steps, alternating model order. Synchronize accelerator work at interval boundaries when using an accelerator. Report median milliseconds per update and valid targets per second. Also time inference with gradients disabled and explicitly state sequence length and batch size. The matching matrix counts control one source of difference; the activation, featurewise product, and kernel shapes still affect runtime.